

# claude-windows / bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f

## Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f\subagents\workflows\wf_b8703ccc-78b\agent-a744e10fcb4695518.jsonl`  
- SHA-256: `99cd0a90e27a8c7367916857f951ef353ea04d5942ade0a863badc1802bba0ba`  
- Source modified: `2026-06-17T14:36:11+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_b8703ccc-78b`  
- session_id: `bf67ec5d-5e54-4f0f-b422-b72d6b2e8a2f`
```

## Transcript

### 1. user (2026-06-17T14:35:18.779Z)

Adversarially VERIFY this code-review finding against the ACTUAL code in undefined. Default to is\_real=false unless you can point to the exact line that proves the bug is real and reachable. Finding: severity=major; title="Renamed variable proc\_src?proc\_detect preserves logic but changes data flow semantics"; where=tools/generate\_highlights.py:149-161; why=The variable proc\_src is renamed to proc\_detect throughout the detection pipeline. When detect\_from\_paths=None, proc\_detect = proc\_src in behavior (same localization logic, same final value). However, the new stitch\_src = src if detect\_from\_paths else proc\_detect assignment (line 182) differs from master's direct use of proc\_src: the old code used proc\_src (the localized detection input) for stitching, but the new code uses proc\_detect (which is identical in the legacy path). This is semantically equivalent but introduces an extra variable assignment that, if later modified, could cause a regression.. Read the relevant file(s) and the tests. Is this a REAL, reachable problem in THIS diff (not a pre-existing issue, not a hypothetical)?

### 2. assistant (2026-06-17T14:35:20.446Z)

I'll analyze this code review finding by examining the actual code in the repository. Let me start by exploring the structure and reading the relevant file.

### 3. user (2026-06-17T14:35:21.119Z)

```
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/pr-stitch
```

### 4. assistant (2026-06-17T14:35:22.386Z)

Now let me read the file to examine the code in question:

### 5. user (2026-06-17T14:35:23.098Z)

```
1      """Batch highlight-reel runner ? orchestrates "generate highlights for these videos  
2      into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.  
3  
4      Why this exists (lessons paid for the hard way):  
5      - **Never delete other work.** Reels + per-video run.logs are written to a LOCAL work  
dir  
6      first, then COPIED into the destination (atomically: temp name -> os.replace). The  
7      destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a  
8      Google-Drive-synced folder once lost finished reels + logs.)  
9      - **Drive-sync safe.** Generating directly inside a Google Drive folder races the sync  
10     client (deleted files resurrect, partial writes sync). Generate locally; publish once.  
11     - **Idempotent re-runs.** Each video's prior DB segments are cleared before processing  
12     (add_video is an UPSERT and does NOT clear children), so a re-run never accumulates  
13     duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).  
14     - **Resumable & concurrency-friendly.** A video whose reel already exists in the
```

```

15     destination is skipped (unless --force); launching a second request never undoes the
16     first's published reels. The Gemini path also isolates its chunk workdir per video
17     (PR for output/chunks/<video_id>) so concurrent videos don't collide.
18
19 Usage:
20     python -m tools.generate_highlights --videos-dir "G:\\...\\June 12 2026" \
21         --out "G:\\...\\June 12 2026\\Highlights" --segmenter gemini [--force]
22     python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
23     """
24     from __future__ import annotations
25
26     import argparse
27     import logging
28     import os
29     import shutil
30     import sys
31     import traceback
32     from typing import Any, Dict, List, Optional, cast
33
34     from dotenv import load_dotenv
35
36     sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
37
38     from backend.config import load_config
39     from backend.infrastructure.database import Database
40     from backend.pipeline.segmenters.base import segmenter_registry
41     from backend.pipeline.stitcher.compiler import VideoCompiler
42     from backend.results import Failure
43     from backend.utils.hashing import compute_video_id
44
45     logger = logging.getLogger("highlights_runner")
46
47     # Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.
48     _WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
49     _VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")
50
51
52     def _atomic_publish(local_path: str, dest_path: str) -> None:
53         """Copy local_path -> dest_path so the destination never sees a partial file.
54
55         Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
56         anything else in the destination folder.
57         """
58         os.makedirs(os.path.dirname(dest_path), exist_ok=True)
59         tmp = dest_path + ".publishing.tmp"
60         shutil.copy2(local_path, tmp)
61         os.replace(tmp, dest_path)
62
63
64     def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
65                                 local_work_dir: str, db_path: str, force: bool = False,
66                                 skip_ai_handoff: bool = False, wasb_stream: bool = False,
67                                 detect_from_paths: Optional[List[str]] = None) -> dict:
68         """Generate one highlight reel per input video into out_dir. Returns a summary dict.
69
70         Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
71
72         skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini
73         handoff);
74             candidate windows become the rallies. wasb_stream: have the WASB detector decode
75             the
76             video on the fly instead of dumping ~46k PNGs (output-preserving fast path,
77             #178).
78         detect_from_paths: optional list parallel to video_paths. When given, the detector
79         runs on

```

```

76         detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source)
but the
77         reel is stitched from video_paths[i] at full resolution. Rally timestamps
transfer 1:1
78         (a pure temporal-preserving downscale shares the timeline). None = detect+stitch
the
79         same source (legacy behaviour).
80         """
81         os.makedirs(out_dir, exist_ok=True)
82         os.makedirs(local_work_dir, exist_ok=True)
83         needs_local = segmenter in _WSL_DETECTORS
84
85         # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is
invoked
86         # standalone ? backend.main does this, but this module is its own entrypoint.
Without it the
87         # AI segmenter finds no API key and silently produces zero rallies. The fully-local
path
88         # (skip_ai_handoff) needs no key, so a missing .env is harmless there.
89         load_dotenv()
90
91         cfg = load_config()
92         # Per-run overrides (typed config; the segmenter reads these via its dict-compat
.get()).
93         # Logged by the segmenter itself once logging is configured below.
94         if skip_ai_handoff:
95             cfg.indexing.skip_ai_handoff = True
96         if wasb_stream:
97             cfg.indexing.wasb.stream_video = True
98         db = Database(db_path)
99         seg_cls = segmenter_registry.get(segmenter)
100         if seg_cls is None:
101             raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")
102
103         # Combined batch log (local; published at the end).
104         fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s",
"%H:%M:%S")
105         if not logging.getLogger().handlers:
106             logging.basicConfig(level=logging.INFO,
handlers=[logging.StreamHandler(sys.stdout)])
107         for h in logging.getLogger().handlers:
108             h.setFormatter(fmt)
109         batch_log_local = os.path.join(local_work_dir, "_batch_run.log")
110         batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
111         batch_fh.setFormatter(fmt)
112         logging.getLogger().addHandler(batch_fh)
113
114         summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [],
"failed": []}
115         logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
116                     len(video_paths), out_dir, segmenter, force)
117
118         for i, src in enumerate(video_paths, 1):
119             name = os.path.basename(src)
120             stem = os.path.splitext(name)[0]
121             reel_name = f"{stem} - Highlights.mp4"
122             dest_reel = os.path.join(out_dir, reel_name)
123             local_reel = os.path.join(local_work_dir, reel_name)
124             # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
125             # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
126             detect_src = detect_from_paths[i - 1] if detect_from_paths else src
127
128             if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
129                 logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),

```

```

name)
130         summary["skipped"].append(name)
131         continue
132     if not os.path.exists(src) or not os.path.exists(detect_src):
133         logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
134                         i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
135         summary["failed"].append(name)
136         continue
137
138     run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
139     run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
140     run_fh.setFormatter(fmt)
141     logging.getLogger().addHandler(run_fh)
142     staged_src: Optional[str] = None
143     try:
144         logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
145         # Localize the DETECTION source for WSL detectors (the WSL FS can't read
Drive/UNC
146         # via /mnt). The stitch source (`src`) is read by ffmpeg directly during
compile, so
147         # only the detector input ever needs staging.
148         if needs_local:
149             staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
150             if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
151                 logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
152                 shutil.copy2(detect_src, staged_src)
153                 proc_detect = staged_src
154             else:
155                 proc_detect = detect_src
156
157         vid = compute_video_id(proc_detect)
158         db.clear_video_data(vid)          # idempotent: no cross-run accumulation
159         db.add_video(vid, proc_detect, "processed", [])
160         if detect_src != src:
161             logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
os.path.basename(detect_src), name)
162             logger.info("video_id=%s segmenting (%s)...", vid, segmenter)
163
164         # AppConfig is consumed by segmenters via its dict-compatible .get shim (same as
165         # main.py); cast keeps the type checker honest about that contract.
166         result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid,
proc_detect)
167
168         if isinstance(result, Failure):
169             logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
170             summary["failed"].append(name)
171             continue
172
173         segs = db.query_play_segments(vid, {})
174         pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
175         logger.info("rallies detected: %d", len(pairs))
176         if not pairs:
177             logger.warning("no rallies -> no reel")
178             summary["no_rallies"].append(name)
179             continue
180
181         # Stitch source: the original `src` when detecting on a separate proxy (hi-
res reel);
182         # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
183         stitch_src = src if detect_from_paths else proc_detect

```

```

184         # Honor the configured branding watermark + segment paddings (stitching.*).
Read
185         # dict-compatibly so a plain-dict config (tests) and the typed AppConfig
both work.
186         # NOTE: the runner previously dropped these ? reels shipped UN-watermarked
with the
187         # ctor default paddings; this threads config.json through (the on-by-default
mark).
188         st = cfg.get("stitching") or {}
189         VideoCompiler(
190             local_work_dir,
191             begin_padding=float(st.get("begin_padding", 0.5)),
192             end_padding=float(st.get("end_padding", 0.5)),
193             watermark=st.get("watermark"),
194         ).compile_highlights(stitch_src, pairs, reel_name)
195         _atomic_publish(local_reel, dest_reel)
196         try:
197             os.remove(local_reel)
198         except OSError:
199             pass
200         logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
201         summary["done"].append(name)
202     except Exception:
203         logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
204         summary["failed"].append(name)
205     finally:
206         logging.getLogger().removeHandler(run_fh)
207         run_fh.close()
208         # publish this video's run.log (additive; never deletes anything in out_dir)
209         try:
210             _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} -
run.log"))
211         except OSError:
212             pass
213         if staged_src and os.path.exists(staged_src):
214             try:
215                 os.remove(staged_src)
216             except OSError:
217                 pass
218
219         logging.getLogger().removeHandler(batch_fh)
220         batch_fh.close()
221         try:
222             _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
223         except OSError:
224             pass
225         logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
226         return summary
227
228
229     def _resolve_videos(args) -> List[str]:
230         if args.video:
231             return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v
in args.video]
232         if args.videos_dir:
233             return sorted(
234                 os.path.join(args.videos_dir, f) for f in os.listdir(args.videos_dir)
235                 if f.endswith(_VIDEO_EXTS) and os.path.isfile(os.path.join(args.videos_dir,
f))
236             )
237         raise SystemExit("provide --videos-dir or one/more --video")
238
239
240     def _resolve_detect_from(args, video_paths: List[str]) -> Optional[List[str]]:
241         """Resolve --detect-from into a list parallel to video_paths (or None if unused).

```

```

242
243     Requires explicit --video (not --videos-dir) and a 1:1 count match ? each detection
244     proxy maps to exactly one --video by position.""
245     if not getattr(args, "detect_from", None):
246         return None
247     if not args.video:
248         raise SystemExit("--detect-from requires explicit --video (not --videos-dir)")
249     if len(args.detect_from) != len(args.video):
250         raise SystemExit(f"--detect-from count ({len(args.detect_from)}) must match "
251                          f"--video count ({len(args.video)}); pair them 1:1 in order")
252     return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in
args.detect_from]
253
254
255     def main() -> None:
256         ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
257         ap.add_argument("--videos-dir", help="folder of source videos (all videos in it,
unless --video given)")
258         ap.add_argument("--video", action="append", help="explicit source video
(repeatable)")
259         ap.add_argument("--out", required=True, help="destination folder for reels +
run.logs")
260         ap.add_argument("--segmenter", default="gemini", help="segmenter registry key
(default: gemini)")
261         ap.add_argument("--local-work", default=os.path.join("output", "highlights_work"),
262                          help="local staging dir (reels compiled here, then published)")
263         ap.add_argument("--db", default=os.path.join("output", "sports_indexer.db"))
264         ap.add_argument("--force", action="store_true", help="regenerate even if a reel
already exists")
265         ap.add_argument("--skip-ai-handoff", action="store_true",
266                          help="FULLY-LOCAL: trajectory-hybrid segmenters emit candidate
windows as "
267                               "rallies with NO Gemini handoff (no API key / upload). Ignored
by --segmenter gemini.")
268         ap.add_argument("--wasb-stream", action="store_true",
269                          help="WASB decode-on-the-fly fast path (#178; output-preserving),
instead of "
270                               "dumping ~46k PNGs. Only affects wasb_hybrid.")
271         ap.add_argument("--detect-from", action="append",
272                          help="detection source ? e.g. a 1080p proxy. The detector analyzes
THIS "
273                               "(cheaper to decode) while the reel is stitched from the
matching --video "
274                               "at full resolution (e.g. the 4K original). Repeat to pair 1:1
with --video "
275                               "in the same order. Timestamps transfer 1:1 (pure temporal
downscale).")
276         args = ap.parse_args()
277         video_paths = _resolve_videos(args)
278         detect_from_paths = _resolve_detect_from(args, video_paths)
279         generate_highlights_batch(video_paths, args.out, segmenter=args.segmenter,
280                                  local_work_dir=args.local_work, db_path=args.db,
force=args.force,
281                                  skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream,
282                                  detect_from_paths=detect_from_paths)
283
284
285     if __name__ == "__main__":
286         main()
287

```

Now let me examine the git history to understand what the "master" code looked like before this change:

## 7. user (2026-06-17T14:35:25.547Z)

```
a6adcc5 fix(runner): honor stitching watermark/padding + add --detect-from for hi-res reels
f5f5e52 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF) (#203)
ef60036 feat(config): R1 milestone ? flip windowing defaults to cap=6 + R4 [DO NOT MERGE w/o
product-go]
8a45334 feat(eval): R5 ? blob-fallback splits the continuously-active blob (default-OFF)
8c68143 feat(eval): R1 ? net over-seg promotion guard unblocks the cap6+R4 milestone (#201)
2eaccd5 feat(eval): nightly rally-quality regression tripwire (golden corpus, CPU) (#202)
a4d8c47 feat(eval): nightly rally-quality regression tripwire (golden corpus, CPU)
0740c66 feat(eval): R1 ? net over-seg promotion guard unblocks the cap6+R4 milestone
34b9caa feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding
(#200)
e248d2d feat(eval): R4 ? rally-END inactivity trim (default-OFF) + R3 cap re-scope finding
7e8f88f feat(eval): R2 ? task-faithful tolerance-match rally metric (augments tIoU) (#199)
a450c53 feat(eval): R2 ? task-faithful tolerance-match rally metric (augments tIoU)
cf8973e docs(eval): R2 design ? task-faithful tolerance-match rally metric (FOR REVIEW) (#198)
bb7b9e4 docs(eval): lock R2 open decisions per owner review (2026-06-17)
38808f0 docs(eval): R2 design ? task-faithful tolerance-match rally metric (for review)
369e90c docs(eval): golden-labeled video registry (the growing golden-finalized set) (#197)
c33cf1d docs(eval): golden-labeled video registry (the growing golden-finalized set)
c483346 docs(quality): real-footage validation capstone + first ground-truth at-par + dual-eval-
set discipline (#196)
ce7eb47 docs(quality): real-footage validation capstone + first ground-truth at-par + dual-eval-
set discipline
bb4d6e2 feat(eval): Tier 1 (W1.5) ? hard-cap fallback enforces the window cap on continuously-
active trajectories (#195)
```

## 8. assistant (2026-06-17T14:35:26.837Z)

Let me look at the diff for the current branch to see what changed:

## 9. user (2026-06-17T14:35:27.396Z)

```
diff --git a/tools/generate_highlights.py b/tools/generate_highlights.py
index ae9f045..565663a 100644
--- a/tools/generate_highlights.py
+++ b/tools/generate_highlights.py
@@ -63,7 +63,8 @@ def _atomic_publish(local_path: str, dest_path: str) -> None:

     def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                                   local_work_dir: str, db_path: str, force: bool = False,
                                   skip_ai_handoff: bool = False, wasb_stream: bool = False) ->
dict:
+
+         skip_ai_handoff: bool = False, wasb_stream: bool = False,
+         detect_from_paths: Optional[List[str]] = None) -> dict:
+         """Generate one highlight reel per input video into out_dir. Returns a summary dict.

+         Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.
@@ -71,6 +72,11 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
+         skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini handoff);
+         candidate windows become the rallies. wasb_stream: have the WASB detector decode the
+         video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).
+         detect_from_paths: optional list parallel to video_paths. When given, the detector runs on
+         detect_from_paths[i] (a cheaper proxy ? e.g. a 1080p downscale of a 4K source) but the
+         reel is stitched from video_paths[i] at full resolution. Rally timestamps transfer 1:1
+         (a pure temporal-preserving downscale shares the timeline). None = detect+stitch the
+         same source (legacy behaviour).
+         """
+         os.makedirs(out_dir, exist_ok=True)
```

```

    os.makedirs(local_work_dir, exist_ok=True)
@@ -115,13 +121,17 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
    reel_name = f"{stem} - Highlights.mp4"
    dest_reel = os.path.join(out_dir, reel_name)
    local_reel = os.path.join(local_work_dir, reel_name)
+    # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
+    # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
+    detect_src = detect_from_paths[i - 1] if detect_from_paths else src

    if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
        logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
        summary["skipped"].append(name)
        continue
-    if not os.path.exists(src):
-        logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
+    if not os.path.exists(src) or not os.path.exists(detect_src):
+        logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
detect_exists=%s)",
+            i, len(video_paths), name, os.path.exists(src),
os.path.exists(detect_src))
        summary["failed"].append(name)
        continue

@@ -132,23 +142,29 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
    staged_src: Optional[str] = None
    try:
        logger.info("[%d/%d] == %s ==", i, len(video_paths), name)
+        # Localize the DETECTION source for WSL detectors (the WSL FS can't read Drive/UNC
+        # via /mnt). The stitch source (`src`) is read by ffmpeg directly during compile,
so
+        # only the detector input ever needs staging.
        if needs_local:
-            staged_src = os.path.join(local_work_dir, name)
-            if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
-                logger.info("localizing source (%.0f MB)...", os.path.getsize(src) / 1024 /
1024)
-                shutil.copy2(src, staged_src)
-                proc_src = staged_src
+            staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
+            if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(detect_src)):
+                logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
+                shutil.copy2(detect_src, staged_src)
+                proc_detect = staged_src
        else:
-            proc_src = src
+            proc_detect = detect_src

-        vid = compute_video_id(proc_src)
+        vid = compute_video_id(proc_detect)
        db.clear_video_data(vid) # idempotent: no cross-run accumulation
-        db.add_video(vid, proc_src, "processed", [])
+        db.add_video(vid, proc_detect, "processed", [])
+        if detect_src != src:
+            logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
+                os.path.basename(detect_src), name)
+            logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

        # AppConfig is consumed by segmenters via its dict-compatible .get shim (same as
        # main.py); cast keeps the type checker honest about that contract.
-        result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)

```



```

+         result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_detect)
+         if isinstance(result, Failure):
+             logger.error("segmentation FAILED: %s", getattr(result, "message", result))
+             summary["failed"].append(name)
@@ -162,8 +178,20 @@ def generate_highlights_batch(video_paths: List[str], out_dir: str, *,
segmenter
+             summary["no_rallies"].append(name)
+             continue

-         # Compile locally, then publish atomically into the destination.
-         VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
+         # Stitch source: the original `src` when detecting on a separate proxy (hi-res
reel);
+         # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
+         stitch_src = src if detect_from_paths else proc_detect
+         # Honor the configured branding watermark + segment paddings (stitching.*). Read
+         # dict-compatibly so a plain-dict config (tests) and the typed AppConfig both work.
+         # NOTE: the runner previously dropped these ? reels shipped UN-watermarked with the
+         # ctor default paddings; this threads config.json through (the on-by-default mark).
+         st = cfg.get("stitching") or {}
+         VideoCompiler(
+             local_work_dir,
+             begin_padding=float(st.get("begin_padding", 0.5)),
+             end_padding=float(st.get("end_padding", 0.5)),
+             watermark=st.get("watermark"),
+         ).compile_highlights(stitch_src, pairs, reel_name)
+         _atomic_publish(local_reel, dest_reel)
+         try:
+             os.remove(local_reel)
@@ -209,6 +237,21 @@ def _resolve_videos(args) -> List[str]:
+         raise SystemExit("provide --videos-dir or one/more --video")

+def _resolve_detect_from(args, video_paths: List[str]) -> Optional[List[str]]:
+    """Resolve --detect-from into a list parallel to video_paths (or None if unused).
+
+    Requires explicit --video (not --videos-dir) and a 1:1 count match ? each detection
+    proxy maps to exactly one --video by position."""
+    if not getattr(args, "detect_from", None):
+        return None
+    if not args.video:
+        raise SystemExit("--detect-from requires explicit --video (not --videos-dir)")
+    if len(args.detect_from) != len(args.video):
+        raise SystemExit(f"--detect-from count ({len(args.detect_from)}) must match "
+            f"--video count ({len(args.video)}); pair them 1:1 in order")
+    return [v if os.path.isabs(v) else os.path.join(args.videos_dir or ".", v) for v in
args.detect_from]
+
+def main() -> None:
+    ap = argparse.ArgumentParser(description="Batch highlight-reel runner (clean, non-
destructive).")
+    ap.add_argument("--videos-dir", help="folder of source videos (all videos in it, unless
--video given)")
@@ -225,10 +268,18 @@ def main() -> None:
+    ap.add_argument("--wasb-stream", action="store_true",
+        help="WASB decode-on-the-fly fast path (#178; output-preserving), instead
of "
+        "dumping ~46k PNGs. Only affects wasb_hybrid.")
+    ap.add_argument("--detect-from", action="append",
+        help="detection source ? e.g. a 1080p proxy. The detector analyzes THIS "
+        "(cheaper to decode) while the reel is stitched from the matching
--video "
+        "at full resolution (e.g. the 4K original). Repeat to pair 1:1 with

```

```
--video "
+
+           "in the same order. Timestamps transfer 1:1 (pure temporal
downscale).")
    args = ap.parse_args()
-   generate_highlights_batch(_resolve_videos(args), args.out, segmenter=args.segmenter,
+   video_paths = _resolve_videos(args)
+   detect_from_paths = _resolve_detect_from(args, video_paths)
+   generate_highlights_batch(video_paths, args.out, segmenter=args.segmenter,
+                             local_work_dir=args.local_work, db_path=args.db,
force=args.force,
-
-           skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream)
+
+           skip_ai_handoff=args.skip_ai_handoff,
wasb_stream=args.wasb_stream,
+
+           detect_from_paths=detect_from_paths)

if __name__ == "__main__":
```

## 10. assistant (2026-06-17T14:35:28.894Z)

Now let me look at the master branch code to understand the original logic:

## 11. user (2026-06-17T14:35:29.563Z)

""Batch highlight-reel runner ? orchestrates "generate highlights for these videos into this folder" requests with CLEAN, NON-DESTRUCTIVE, resumable execution.

Why this exists (lessons paid for the hard way):

- **\*\*Never delete other work.\*\*** Reels + per-video run.logs are written to a LOCAL work dir first, then COPIED into the destination (atomically: temp name -> os.replace). The destination folder is only ever ADDED to ? we never bulk-delete it. (A bulk `rm` of a Google-Drive-synced folder once lost finished reels + logs.)
- **\*\*Drive-sync safe.\*\*** Generating directly inside a Google Drive folder races the sync client (deleted files resurrect, partial writes sync). Generate locally; publish once.
- **\*\*Idempotent re-runs.\*\*** Each video's prior DB segments are cleared before processing (add\_video is an UPSERT and does NOT clear children), so a re-run never accumulates duplicate rallies. Belt-and-suspenders with VideoCompiler's overlap-merge (PR #180).
- **\*\*Resumable & concurrency-friendly.\*\*** A video whose reel already exists in the destination is skipped (unless --force); launching a second request never undoes the first's published reels. The Gemini path also isolates its chunk workdir per video (PR for output/chunks/<video\_id>) so concurrent videos don't collide.

Usage:

```
python -m tools.generate_highlights --videos-dir "G:\...\June 12 2026" \
    --out "G:\...\June 12 2026\Highlights" --segmenter gemini [--force]
python -m tools.generate_highlights --video A.MP4 --video B.MP4 --out <dir> ...
"""
```

```
from __future__ import annotations
```

```
import argparse
import logging
import os
import shutil
import sys
import traceback
from typing import Any, Dict, List, Optional, cast
```

```
from dotenv import load_dotenv
```

```
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
```

```
from backend.config import load_config
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
```

```

from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.results import Failure
from backend.utils.hashing import compute_video_id

```

```

logger = logging.getLogger("highlights_runner")

```

## Detectors that run in WSL can't read Google Drive / UNC via /mnt -> localize first.

```

_WSL_DETECTORS = {"wasb_hybrid", "tracknet_hybrid", "yolo_hybrid"}
_VIDEO_EXTS = (".mp4", ".MP4", ".mov", ".MOV", ".mkv", ".avi")

```

```

def _atomic_publish(local_path: str, dest_path: str) -> None:
    """Copy local_path -> dest_path so the destination never sees a partial file.

```

```

    Copy to a temp sibling, then os.replace (atomic on the same volume). Never deletes
    anything else in the destination folder.

```

```

    """

```

```

    os.makedirs(os.path.dirname(dest_path), exist_ok=True)
    tmp = dest_path + ".publishing.tmp"
    shutil.copy2(local_path, tmp)
    os.replace(tmp, dest_path)

```

```

def generate_highlights_batch(video_paths: List[str], out_dir: str, *, segmenter: str,
                              local_work_dir: str, db_path: str, force: bool = False,
                              skip_ai_handoff: bool = False, wasb_stream: bool = False) -> dict:
    """Generate one highlight reel per input video into out_dir. Returns a summary dict.

```

```

    Non-destructive: only ADDS reels/logs to out_dir (atomic publish); never deletes.

```

```

    skip_ai_handoff: run the trajectory-hybrid segmenters fully-local (no Gemini handoff);
    candidate windows become the rallies. wasb_stream: have the WASB detector decode the
    video on the fly instead of dumping ~46k PNGs (output-preserving fast path, #178).

```

```

    """

```

```

    os.makedirs(out_dir, exist_ok=True)
    os.makedirs(local_work_dir, exist_ok=True)
    needs_local = segmenter in _WSL_DETECTORS

```

```

    # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is invoked
    # standalone ? backend.main does this, but this module is its own entrypoint. Without it the
    # AI segmenter finds no API key and silently produces zero rallies. The fully-local path
    # (skip_ai_handoff) needs no key, so a missing .env is harmless there.
    load_dotenv()

```

```

    cfg = load_config()
    # Per-run overrides (typed config; the segmenter reads these via its dict-compatible .get).
    # Logged by the segmenter itself once logging is configured below.
    if skip_ai_handoff:
        cfg.indexing.skip_ai_handoff = True
    if wasb_stream:
        cfg.indexing.wasb.stream_video = True
    db = Database(db_path)
    seg_cls = segmenter_registry.get(segmenter)
    if seg_cls is None:
        raise SystemExit(f"unknown segmenter '{segmenter}'; available:
{segmenter_registry.list_available()}")

```

```

    # Combined batch log (local; published at the end).

```

```

    fmt = logging.Formatter("%(asctime)s [%(levelname)s] %(name)s: %(message)s", "%H:%M:%S")

```

```

    if not logging.getLogger().handlers:

```

```

        logging.basicConfig(level=logging.INFO, handlers=[logging.StreamHandler(sys.stdout)])

```

```

        for h in logging.getLogger().handlers:

```

```

            h.setFormatter(fmt)

```

```

    batch_log_local = os.path.join(local_work_dir, "_batch_run.log")

```

```

batch_fh = logging.FileHandler(batch_log_local, mode="w", encoding="utf-8")
batch_fh.setFormatter(fmt)
logging.getLogger().addHandler(batch_fh)

summary: Dict[str, List[str]] = {"done": [], "skipped": [], "no_rallies": [], "failed": []}
logger.info("highlights runner: %d videos -> %s (segmenter=%s, force=%s)",
            len(video_paths), out_dir, segmenter, force)

for i, src in enumerate(video_paths, 1):
    name = os.path.basename(src)
    stem = os.path.splitext(name)[0]
    reel_name = f"{stem} - Highlights.mp4"
    dest_reel = os.path.join(out_dir, reel_name)
    local_reel = os.path.join(local_work_dir, reel_name)

    if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
        logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
        summary["skipped"].append(name)
        continue
    if not os.path.exists(src):
        logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
        summary["failed"].append(name)
        continue

    run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
    run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
    run_fh.setFormatter(fmt)
    logging.getLogger().addHandler(run_fh)
    staged_src: Optional[str] = None
    try:
        logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
        if needs_local:
            staged_src = os.path.join(local_work_dir, name)
            if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
                logger.info("localizing source (%.0f MB)...", os.path.getsize(src) / 1024 /
1024)
                shutil.copy2(src, staged_src)
            proc_src = staged_src
        else:
            proc_src = src

        vid = compute_video_id(proc_src)
        db.clear_video_data(vid) # idempotent: no cross-run accumulation
        db.add_video(vid, proc_src, "processed", [])
        logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

        # AppConfig is consumed by segmenters via its dict-compatible .get shim (same as
        # main.py); cast keeps the type checker honest about that contract.
        result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
        if isinstance(result, Failure):
            logger.error("segmentation FAILED: %s", getattr(result, "message", result))
            summary["failed"].append(name)
            continue

        segs = db.query_play_segments(vid, {})
        pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
        logger.info("rallies detected: %d", len(pairs))
        if not pairs:
            logger.warning("no rallies -> no reel")
            summary["no_rallies"].append(name)
            continue

        # Compile locally, then publish atomically into the destination.
        VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)

```

```

        _atomic_publish(local_reel, dest_reel)
    try:
        os.remove(local_reel)
    except OSError:
        pass
    logger.info("PUBLISHED reel: %s (%d rallies)", dest_reel, len(pairs))
    summary["done"].append(name)
except Exception:
    logger.error("ERROR on %s:\n%s", name, traceback.format_exc())
    summary["failed"].append(name)
finally:
    logging.getLogger().removeHandler(run_fh)
    run_fh.close()
    # publish this video's run.log (additive; never deletes anything in out_dir)
    try:
        _atomic_publish(run_log_local, os.path.join(out_dir, f"{stem} - run.log"))
    except OSError:
        pass
    if staged_src and os.path.exists(staged_src):
        try:
            os.remove(staged_src)
        except OSError:
            pass

logging.getLogger().removeHandler(batch_fh)
batch_fh.close()
try:
    _atomic_publish(batch_log_local, os.path.join(out_dir, "_batch_run.log"))
except OSError:
    pass
logger.info("DONE. %s", {k: len(v) for k, v in summary.items()})
return summary

```

## 12. assistant (2026-06-17T14:35:30.986Z)

Now let me check for tests that validate this behavior:

## 13. user (2026-06-17T14:35:31.971Z)

```

C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_generate_highlights.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-
a771ff1d85e18f2ed/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/boxhill17/tests/test_generate_highlights.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/docs-
golden/tests/test_generate_highlights.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/docs-rally-
quality/tests/test_generate_highlights.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/docs-
today/tests/test_generate_highlights.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/hardchop/tests/test_generate_highlights.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/hardchop/tests/__pycache__/
test_generate_highlights.cpython-313-pytest-9.0.3.pyc
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/local-
noai/tests/test_generate_highlights.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/local-
noai/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/pr-
stitch/tests/test_generate_highlights.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/pr-
stitch/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/r2-impl/tests/test_generate_highlights.py

```

```

C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/r2-impl/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/r4-endrule/tests/test_generate_highlights.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/r4-endrule/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/tier0-eval/tests/test_generate_highlights.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/tier0-eval/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/tier1-windowing/tests/test_generate_highlights.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/tier1-windowing/tests/__pycache__/test_generate_highlights.cpython-313-pytest-9.0.3.pyc

```

#### 14. user (2026-06-17T14:35:33.581Z)

```

1      """tools/generate_highlights.py ? the batch highlight runner. The load-bearing
guarantees
2      are NON-DESTRUCTIVE + atomic publish + idempotent skip; those are what we lock here
3      (heavy deps ? segmenter/compiler/db ? are mocked)."""
4      import os
5      from unittest.mock import MagicMock, patch
6
7      import pytest
8
9      import tools.generate_highlights as gh
10
11
12      def _fake_compiler(captured):
13          """A stand-in VideoCompiler that records its ctor kwargs + compile args and writes
the
14          expected reel file (so the runner's atomic-publish step succeeds)."""
15          class FakeCompiler:
16              def __init__(self, output_dir, end_padding=1.0, begin_padding=0.5,
watermark=None):
17                  captured["ctor"] = dict(output_dir=output_dir, end_padding=end_padding,
18                                          begin_padding=begin_padding, watermark=watermark)
19
20              def compile_highlights(self, raw_video_path, segments, output_filename):
21                  captured["compile"] = dict(raw=raw_video_path, segments=list(segments),
22                                          out=output_filename)
23                  reel = os.path.join(captured["ctor"]["output_dir"], output_filename)
24                  with open(reel, "wb") as fh:
25                      fh.write(b"reel-bytes")
26                  return reel
27
28          return FakeCompiler
29
30
31      def test_atomic_publish_is_nondestructive(tmp_path):
32          src = tmp_path / "local_reel.mp4"
33          src.write_bytes(b"reel-bytes")
34          dest_dir = tmp_path / "out"
35          dest_dir.mkdir()
36          sibling = dest_dir / "Someone Else - Highlights.mp4"
37          sibling.write_bytes(b"do-not-touch")
38
39          gh._atomic_publish(str(src), str(dest_dir / "Me - Highlights.mp4"))
40
41          assert (dest_dir / "Me - Highlights.mp4").read_bytes() == b"reel-bytes"
42          assert sibling.read_bytes() == b"do-not-touch"          # sibling untouched
43          assert not list(dest_dir.glob("*.publishing.tmp"))      # no partial left behind
44
45

```

```

46     def test_resolve_videos_explicit_and_dir(tmp_path):
47         (tmp_path / "a.MP4").write_bytes(b"x")
48         (tmp_path / "b.mp4").write_bytes(b"x")
49         (tmp_path / "notes.txt").write_text("ignore")
50         args = MagicMock(video=None, videos_dir=str(tmp_path))
51         found = gh._resolve_videos(args)
52         assert [os.path.basename(p) for p in found] == ["a.MP4", "b.mp4"] # videos only,
sorted
53         args2 = MagicMock(video=["only.MP4"], videos_dir=str(tmp_path))
54         assert gh._resolve_videos(args2) == [os.path.join(str(tmp_path), "only.MP4")]
55
56
57     def test_batch_skips_existing_reel_and_preserves_other_work(tmp_path):
58         """force=False: a video whose reel already exists is skipped, the segmenter is never
59         called, and unrelated files already in the destination are left intact."""
60         out = tmp_path / "out"
61         out.mkdir()
62         (out / "Adarsh v Avi - Highlights.mp4").write_bytes(b"already-published")
63         (out / "Adarsh v Avi - run.log").write_text("prior log")
64         other = out / "Someone Else - Highlights.mp4"
65         other.write_bytes(b"untouched")
66         src = tmp_path / "Adarsh v Avi.MP4"
67         src.write_bytes(b"src")
68
69         seg_registry = MagicMock()
70         with patch.object(gh, "load_config", return_value={}), \
71             patch.object(gh, "Database", return_value=MagicMock()), \
72             patch.object(gh, "segmenter_registry", seg_registry):
73             summary = gh.generate_highlights_batch(
74                 [str(src)], str(out), segmenter="gemini",
75                 local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
force=False)
76
77         assert summary["skipped"] == ["Adarsh v Avi.MP4"]
78         # never processed: the resolved segmenter class was never instantiated/run
79         seg_registry.get.return_value.assert_not_called()
80         assert other.read_bytes() == b"untouched" # other work preserved
81         assert (out / "Adarsh v Avi - Highlights.mp4").read_bytes() == b"already-published"
# not regenerated
82
83
84     def test_batch_applies_local_overrides(tmp_path):
85         """--skip-ai-handoff / --wasb-stream inject into the typed config before
segmentation."""
86         from backend.config import AppConfig
87         cfg = AppConfig()
88         assert cfg.indexing.skip_ai_handoff is False # default off
89         assert cfg.indexing.wasb.stream_video is False
90
91         out = tmp_path / "out"
92         out.mkdir()
93         src = tmp_path / "clip.MP4"
94         src.write_bytes(b"src")
95
96         seg_instance = MagicMock()
97         seg_instance.process_video.return_value = [] # no rallies -> no
compile/publish
98         seg_registry = MagicMock()
99         seg_registry.get.return_value = MagicMock(return_value=seg_instance)
100
101         with patch.object(gh, "load_config", return_value=cfg), \
102             patch.object(gh, "Database", return_value=MagicMock()), \
103             patch.object(gh, "compute_video_id", return_value="vid123"), \
104             patch.object(gh, "segmenter_registry", seg_registry):
105             gh.generate_highlights_batch(

```

```

106         [str(src)], str(out), segmenter="gemini",
107         local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
108         force=True, skip_ai_handoff=True, wasb_stream=True)
109
110     assert cfg.indexing.skip_ai_handoff is True
111     assert cfg.indexing.wasb.stream_video is True
112
113
114     def _run_one(tmp_path, *, video_paths, detect_from_paths=None, segmenter="gemini"):
115         """Drive generate_highlights_batch once with a fake compiler/segmenter/db, returning
116         the
117         captured VideoCompiler ctor+compile args and the compute_video_id mock."""
118         from backend.config import AppConfig
119         cfg = AppConfig() # watermark on by default
120         out = tmp_path / "out"
121         out.mkdir()
122         db = MagicMock()
123         db.query_play_segments.return_value = [{"start_time": 1.0, "end_time": 5.0}]
124         seg_instance = MagicMock()
125         seg_instance.process_video.return_value = ["ok"] # non-Failure
126         seg_registry = MagicMock()
127         seg_registry.get.return_value = MagicMock(return_value=seg_instance)
128         captured: dict = {}
129         cvid = MagicMock(return_value="vid123")
130         with patch.object(gh, "load_config", return_value=cfg), \
131             patch.object(gh, "Database", return_value=db), \
132             patch.object(gh, "compute_video_id", cvid), \
133             patch.object(gh, "VideoCompiler", _fake_compiler(captured)), \
134             patch.object(gh, "segmenter_registry", seg_registry):
135             gh.generate_highlights_batch(
136                 [str(p) for p in video_paths], str(out), segmenter=segmenter,
137                 local_work_dir=str(tmp_path / "work"), db_path=str(tmp_path / "db.sqlite"),
138                 force=True, detect_from_paths=[str(p) for p in detect_from_paths] if
139                 detect_from_paths else None)
140             return captured, cvid, out
141
142     def test_batch_threads_watermark_and_padding_into_compiler(tmp_path):
143         """REGRESSION: the runner must pass stitching.watermark + begin/end_padding to
144         VideoCompiler
145         (it previously constructed VideoCompiler bare -> reels shipped un-watermarked)."""
146         src = tmp_path / "clip.MP4"
147         src.write_bytes(b"src")
148         captured, _cvid, out = _run_one(tmp_path, video_paths=[src])
149
150         wm = captured["ctor"]["watermark"]
151         assert wm and wm.get("enabled") is True # on-by-default mark
152
153         threaded through
154         assert wm.get("text") == "Generated by Khelsutra.guru"
155         assert captured["ctor"]["begin_padding"] == 0.5 # config paddings, not the
156         ctor 1.0 default
157         assert captured["ctor"]["end_padding"] == 0.5
158         assert captured["compile"]["raw"] == str(src) # stitches from the (only)
159         source
160         assert (out / "clip - Highlights.mp4").exists() # published
161
162
163     def test_batch_detect_from_proxy_stitches_from_original(tmp_path):
164         """--detect-from: the detector runs on the proxy (video_id keyed off it) but the
165         reel is
166         stitched from the full-res --video, and named after it."""
167         orig = tmp_path / "GX01.MP4"
168         orig.write_bytes(b"original-4k")
169         proxy = tmp_path / "GX01_1080p.mp4"
170         proxy.write_bytes(b"proxy-1080")

```



```

164         captured, cvid, out = _run_one(tmp_path, video_paths=[orig],
detect_from_paths=[proxy])
165
166         cvid.assert_called_once_with(str(proxy))                # detection keyed off the
proxy
167         assert captured["compile"]["raw"] == str(orig)          # reel stitched from the
original
168         assert (out / "GX01 - Highlights.mp4").exists()          # named after the original,
not the proxy
169
170
171     def test_resolve_detect_from_validates_pairing(tmp_path):
172         assert gh._resolve_detect_from(MagicMock(detect_from=None), ["v"]) is None
173         with pytest.raises(SystemExit):                          # count mismatch
174             gh._resolve_detect_from(MagicMock(detect_from=["a", "b"], video=["v1"],
videos_dir=None), ["v1"])
175         with pytest.raises(SystemExit):                          # requires explicit --video
176             gh._resolve_detect_from(MagicMock(detect_from=["a"], video=None,
videos_dir="d"), ["v1"])
177         abs = os.path.join(os.sep, "abs", "p.mp4")
178         assert gh._resolve_detect_from(
179             MagicMock(detect_from=[abs], video=["v1"], videos_dir=None), ["v1"]) == [abs]
180

```

## 15. assistant (2026-06-17T14:35:35.472Z)

Now let me trace through the logic carefully in both the old and new code. Let me create a detailed comparison:

## 16. user (2026-06-17T14:35:36.464Z)

```

if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
    logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths), name)
    summary["skipped"].append(name)
    continue
if not os.path.exists(src):
    logger.warning("[%d/%d] SKIP %s ? source missing", i, len(video_paths), name)
    summary["failed"].append(name)
    continue

run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
run_fh.setFormatter(fmt)
logging.getLogger().addHandler(run_fh)
staged_src: Optional[str] = None
try:
    logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
    if needs_local:
        staged_src = os.path.join(local_work_dir, name)
        if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==
os.path.getsize(src)):
            logger.info("localizing source (%.0f MB)...", os.path.getsize(src) / 1024 /
1024)
            shutil.copy2(src, staged_src)
        proc_src = staged_src
    else:
        proc_src = src

    vid = compute_video_id(proc_src)
    db.clear_video_data(vid)                # idempotent: no cross-run accumulation
    db.add_video(vid, proc_src, "processed", [])
    logger.info("video_id=%s segmenting (%s)...", vid, segmenter)

    # AppConfig is consumed by segmenters via its dict-compatible .get shim (same as
    # main.py); cast keeps the type checker honest about that contract.

```

```

result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid, proc_src)
if isinstance(result, Failure):
    logger.error("segmentation FAILED: %s", getattr(result, "message", result))
    summary["failed"].append(name)
    continue

segs = db.query_play_segments(vid, {})
pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
logger.info("rallies detected: %d", len(pairs))
if not pairs:
    logger.warning("no rallies -> no reel")
    summary["no_rallies"].append(name)
    continue

# Compile locally, then publish atomically into the destination.
VideoCompiler(local_work_dir).compile_highlights(proc_src, pairs, reel_name)
_atomic_publish(local_reel, dest_reel)
try:
    os.remove(local_reel)
except OSError:

```

### 17. assistant (2026-06-17T14:35:38.371Z)

Perfect. Now I have the full picture. Let me verify the exact lines in the new code again:

### 18. user (2026-06-17T14:35:38.748Z)

```

118     for i, src in enumerate(video_paths, 1):
119         name = os.path.basename(src)
120         stem = os.path.splitext(name)[0]
121         reel_name = f"{stem} - Highlights.mp4"
122         dest_reel = os.path.join(out_dir, reel_name)
123         local_reel = os.path.join(local_work_dir, reel_name)
124         # Detection source: a (cheaper) proxy when --detect-from is paired with --video;
125         # otherwise the video itself. The reel is ALWAYS stitched from `src` (full res).
126         detect_src = detect_from_paths[i - 1] if detect_from_paths else src
127
128         if not force and os.path.exists(dest_reel) and os.path.getsize(dest_reel) > 0:
129             logger.info("[%d/%d] SKIP %s ? reel already published", i, len(video_paths),
130 name)
131             summary["skipped"].append(name)
132             continue
133         if not os.path.exists(src) or not os.path.exists(detect_src):
134             logger.warning("[%d/%d] SKIP %s ? source missing (stitch_exists=%s
135 detect_exists=%s)",
136 i, len(video_paths), name, os.path.exists(src),
137 os.path.exists(detect_src))
138             summary["failed"].append(name)
139             continue
140
141         run_log_local = os.path.join(local_work_dir, f"{stem} - run.log")
142         run_fh = logging.FileHandler(run_log_local, mode="w", encoding="utf-8")
143         run_fh.setFormatter(fmt)
144         logging.getLogger().addHandler(run_fh)
145         staged_src: Optional[str] = None
146         try:
147             logger.info("[%d/%d] === %s ===", i, len(video_paths), name)
148             # Localize the DETECTION source for WSL detectors (the WSL FS can't read
149 Drive/UNC
150             # via /mnt). The stitch source (`src`) is read by ffmpeg directly during
151 compile, so
152             # only the detector input ever needs staging.
153             if needs_local:
154                 staged_src = os.path.join(local_work_dir, os.path.basename(detect_src))
155                 if not (os.path.exists(staged_src) and os.path.getsize(staged_src) ==

```

```

os.path.getsize(detect_src)):
151         logger.info("localizing detection source (%.0f MB)...",
os.path.getsize(detect_src) / 1024 / 1024)
152         shutil.copy2(detect_src, staged_src)
153         proc_detect = staged_src
154     else:
155         proc_detect = detect_src
156
157     vid = compute_video_id(proc_detect)
158     db.clear_video_data(vid)          # idempotent: no cross-run accumulation
159     db.add_video(vid, proc_detect, "processed", [])
160     if detect_src != src:
161         logger.info("detecting on proxy '%s' -> stitching reel from '%s'",
162                     os.path.basename(detect_src), name)
163     logger.info("video_id=%s segmenting (%s)...", vid, segmenter)
164
165     # AppConfig is consumed by segmenters via its dict-compatible .get shim (same as
166     # main.py); cast keeps the type checker honest about that contract.
167     result = seg_cls(db, cast(Dict[str, Any], cfg)).process_video(vid,
proc_detect)
168     if isinstance(result, Failure):
169         logger.error("segmentation FAILED: %s", getattr(result, "message",
result))
170         summary["failed"].append(name)
171         continue
172
173     segs = db.query_play_segments(vid, {})
174     pairs = sorted((float(s["start_time"]), float(s["end_time"])) for s in segs)
175     logger.info("rallies detected: %d", len(pairs))
176     if not pairs:
177         logger.warning("no rallies -> no reel")
178         summary["no_rallies"].append(name)
179         continue
180
181     # Stitch source: the original `src` when detecting on a separate proxy (hi-
res reel);
182     # otherwise the (possibly localized) detection input ? identical to legacy
behaviour.
183     stitch_src = src if detect_from_paths else proc_detect
184     # Honor the configured branding watermark + segment paddings (stitching.*).
Read
185     # dict-compatibly so a plain-dict config (tests) and the typed AppConfig
both work.
186     # NOTE: the runner previously dropped these ? reels shipped UN-watermarked
with the
187     # ctor default paddings; this threads config.json through (the on-by-default
mark).
188     st = cfg.get("stitching") or {}
189     VideoCompiler(
190         local_work_dir,
191         begin_padding=float(st.get("begin_padding", 0.5)),
192         end_padding=float(st.get("end_padding", 0.5)),

```

## 19. assistant (2026-06-17T14:35:51.142Z)

Perfect. Now I have all the information I need. Let me analyze the code semantically:

**\*\*MASTER (old) code path:\*\***

- Line 136 (master): `if needs\_local:` -> localizes `src` to `staged\_src`
- Line 140 (master): `proc\_src = staged\_src` or `proc\_src = src`
- Line 142 (master): `vid = compute\_video\_id(proc\_src)`
- Line 165 (master): `VideoCompiler(local\_work\_dir).compile\_highlights(proc\_src, pairs, reel\_name)`

When `detect\_from\_paths=None` (legacy behavior):

- `proc\_src` is either `staged\_src` (localized version of `src`) or `src` itself
- The stitching uses `proc\_src` for the compile call

**\*\*NEW code path:\*\***

- Line 126: `detect\_src = detect\_from\_paths[i - 1] if detect\_from\_paths else src`
- Line 148-155: Localizes `detect\_src` to `staged\_src`
- Line 153/155: `proc\_detect = staged\_src` or `proc\_detect = detect\_src`
- Line 157: `vid = compute\_video\_id(proc\_detect)`
- Line 183: `stitch\_src = src if detect\_from\_paths else proc\_detect`
- Line 194: `VideoCompiler(...).compile\_highlights(stitch\_src, pairs, reel\_name)`

When `detect\_from\_paths=None` (legacy behavior):

- `detect\_src = src` (line 126)
- `proc\_detect` = either `staged\_src` (localized `src`) or `src`
- `stitch\_src = proc\_detect` (since `detect\_from\_paths` is None, line 183)
- VideoCompiler.compile\_highlights is called with `stitch\_src` = `proc\_detect`

Now let me trace through a concrete scenario to verify equivalence. Let me check the test to see how this is validated:

Looking at test line 157-168, the test validates the new behavior with `detect\_from\_paths`. Let me check if there's a test that validates the LEGACY path (when `detect\_from\_paths=None`):

The test at line 141-154 (`test\_batch\_threads\_watermark\_and\_padding\_into\_compiler`) does run with `detect\_from\_paths=None` implicitly (no detect\_from\_paths argument passed to `\_run\_one`). Let me examine what that test checks:

Looking at the test: it calls `\_run\_one(tmp\_path, video\_paths=[src])` without `detect\_from\_paths`, which means `detect\_from\_paths=None`. The test captures the compiler's arguments and checks:

- Line 153: `assert captured["compile"]["raw"] == str(src)`

This confirms the new code works correctly for the legacy case. The test passes because in the new code when `detect\_from\_paths=None`:

- `detect\_src = src` (line 126)
- If `needs\_local` is False: `proc\_detect = detect\_src = src`
- `stitch\_src = src if detect\_from\_paths else proc\_detect = proc\_detect = src`
- So `compile\_highlights(str(src), ...)`

Now let me check the critical point: is there a case where the behavior differs? The finding claims that "the new code uses proc\_detect (which is identical in the legacy path)" but "introduces an extra variable assignment that, if later modified, could cause a regression."

This is the key question: **\*\*Is `stitch\_src = src if detect\_from\_paths else proc\_detect` logically equivalent to the master's direct use of `proc\_src` in all cases?\*\***

Let me trace both paths when `detect\_from\_paths=None` and `needs\_local=True`:

**\*\*MASTER:\*\***

- `proc\_src = staged\_src` (a localized copy of `src`)
- `compile\_highlights(proc\_src, ...)` = `compile\_highlights(staged\_src, ...)`

**\*\*NEW:\*\***

- `proc\_detect = staged\_src` (a localized copy of `detect\_src` which equals `src`)
- `stitch\_src = src if detect\_from\_paths else proc\_detect` = `proc\_detect` = `staged\_src`
- `compile\_highlights(stitch\_src, ...)` = `compile\_highlights(staged\_src, ...)`

These are **\*\*identical\*\***.

When `detect\_from\_paths=None` and `needs\_local=False`:

**\*\*MASTER:\*\***

- `proc\_src = src`
- `compile\_highlights(proc\_src, ...)` = `compile\_highlights(src, ...)`

```
**NEW:**
- `proc_detect = detect_src = src`
- `stitch_src = src if detect_from_paths else proc_detect` = `proc_detect` = `src`
- `compile_highlights(stitch_src, ...)` = `compile_highlights(src, ...)`
```

These are **\*\*also identical\*\***.

Now let me verify the finding's concern is real: "the new code uses proc\_detect (which is identical in the legacy path). This is semantically equivalent but introduces an extra variable assignment that, if later modified, could cause a regression."

The finding is saying this is a POTENTIAL future regression risk, not an actual current bug. Let me check if there's actual logic that's broken TODAY.

Let me run the tests to verify nothing is broken:

## 20. user (2026-06-17T14:35:59.747Z)

```
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\pr-stitch
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 7 items

tests/test_generate_highlights.py::test_atomic_publish_is_nondestructive PASSED [ 14%]
tests/test_generate_highlights.py::test_resolve_videos_explicit_and_dir PASSED [ 28%]
tests/test_generate_highlights.py::test_batch_skips_existing_reel_and_preserves_other_work
PASSED [ 42%]
tests/test_generate_highlights.py::test_batch_applies_local_overrides PASSED [ 57%]
tests/test_generate_highlights.py::test_batch_threads_watermark_and_padding_into_compiler PASSED
[ 71%]
tests/test_generate_highlights.py::test_batch_detect_from_proxy_stitches_from_original PASSED [
85%]
tests/test_generate_highlights.py::test_resolve_detect_from_validates_pairing PASSED [100%]

===== 7 passed in 5.80s =====
```

## 21. assistant (2026-06-17T14:36:06.448Z)

All tests pass. Now let me assess whether the finding identifies a REAL bug in THIS diff:

The finding states:

```
> "The variable proc_src is renamed to proc_detect throughout the detection pipeline. When
detect_from_paths=None, proc_detect = proc_src in behavior (same localization logic, same final
value). However, the new stitch_src = src if detect_from_paths else proc_detect assignment (line
182) differs from master's direct use of proc_src: the old code used proc_src (the localized
detection input) for stitching, but the new code uses proc_detect (which is identical in the
legacy path). This is semantically equivalent but introduces an extra variable assignment that,
if later modified, could cause a regression."
```

The claim is:

1. Master used `proc\_src` for stitching
2. New code uses `stitch\_src` which equals `proc\_detect` in the legacy case
3. These are semantically equivalent NOW
4. But introduces future regression risk

**\*\*Is this a REAL, reachable problem in THIS diff?\*\***

The answer is: **\*\*No, this is not a real problem in THIS diff.\*\***

Here's why:

1. The behavior is demonstrably identical (both paths compile the same file)

2. The tests validate this equivalence and all pass
3. The comment on line 181-182 explicitly documents the equivalence: "identical to legacy behaviour"
4. The extra variable `stitch\_src` is NOT creating a bug today; it's creating clarity by separating concerns:
  - `proc\_detect` = the input to the detector (either staged or not)
  - `stitch\_src` = the input to the compiler (either the original src or the detection input)
5. This is actually BETTER than the master code because it makes the intent explicit

The finding is conflating two different things:

- A **semantic equivalence** (true)
- A **real, reachable bug** (false)

The extra variable assignment (`stitch\_src = src if detect\_from\_paths else proc\_detect`) is not introducing a bug; it's documenting the split between detection source and stitching source. When `detect\_from\_paths` is None, the new code intentionally uses the same source for both detection and stitching, which is correct.

The "regression risk" mentioned is purely hypothetical (if someone later modifies the conditional incorrectly), not an actual problem in the current code.

## **22. user (2026-06-17T14:36:11.330Z)**

Structured output provided successfully